

PATRONES DE DISEÑO

Arquitectura por Capas

Implementación en sistema de
préstamos en Python.



Estructura Jerárquica

Separación de responsabilidades en niveles independientes. Cada nivel solo conoce al inmediatamente inferior.

1. Presentación (UI)



2. Lógica de Negocio



3. Acceso a Datos

Análisis del Patrón



Características

- Flujo de datos unidireccional.
- Dependencias descendentes.
- Responsabilidades aisladas.



Ventajas

- **Bajo acoplamiento:** Cambios contenidos.
- **Alta cohesión:** Código predecible.
- **Testabilidad:** Pruebas unitarias fáciles.



Desventajas

- **Burocracia:** Clases redundantes.
- **Latencia:** Saltos entre múltiples capas.
- **Antipatrones:** Riesgo de capas huecas.

La Analogía del Restaurante

- **El Mesero (Presentación):** Recibe la orden del cliente. No cocina.
- **El Chef (Aplicación):** Ejecuta la receta. Aplica reglas de negocio.
- **La Despensa (Datos):** Almacena ingredientes. No sabe de recetas.



INVERSIÓN DE DEPENDENCIAS (DIP)

“El Chef no va a la granja a cultivar los tomates.”



Detalle de Implementación

Cultivo específico, proveedor local.



Abstracción (Interfaz)

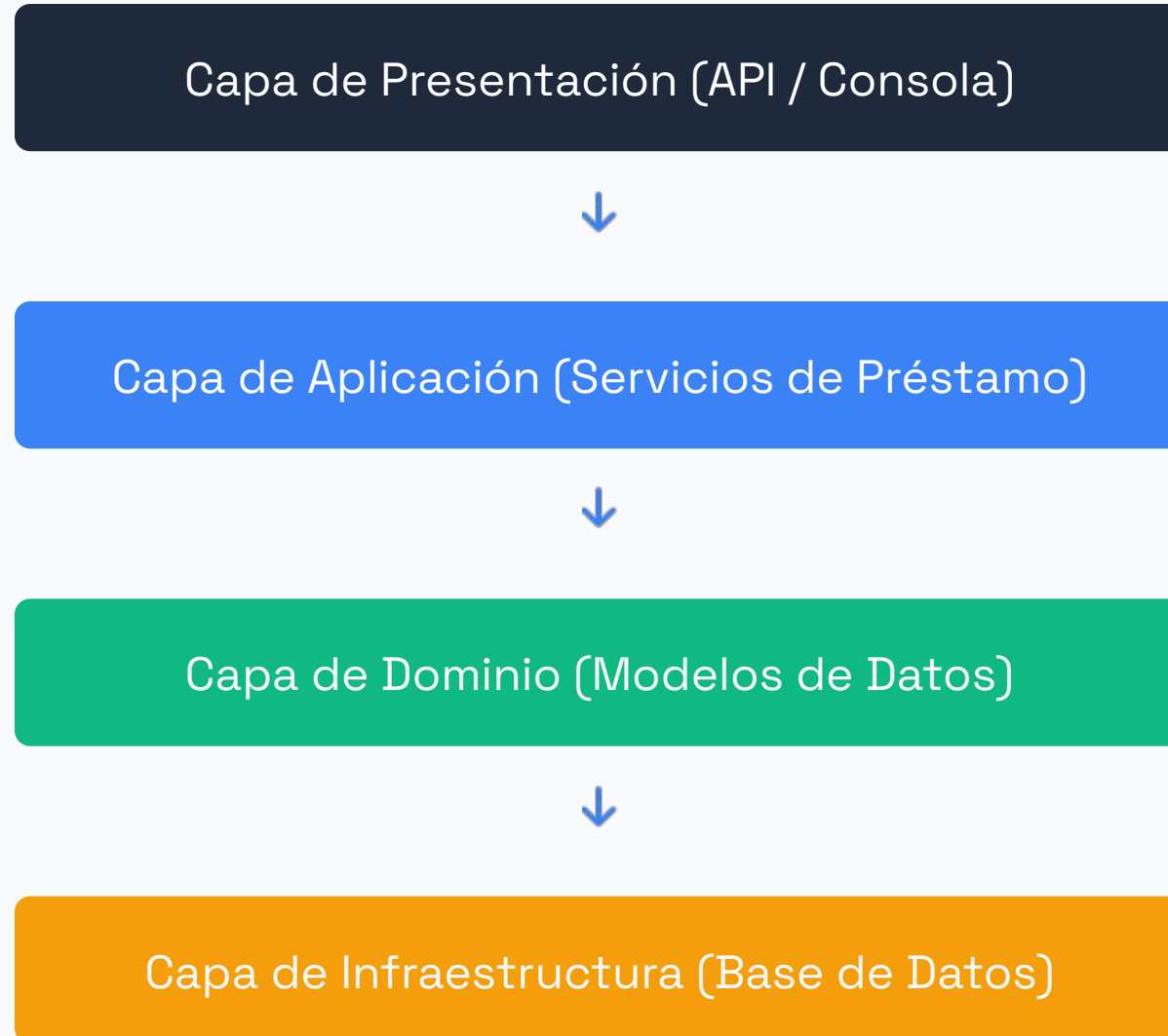
Caja de tomates estándar entregada.

El Problema a resolver

Sistema de Préstamos

El sistema debe recibir solicitudes, verificar si hay equipos en el inventario, aprobar o rechazar la solicitud, y mostrar un reporte final.

Diagrama del Sistema de Préstamos



Estructura de Directorios

src/

presentation/

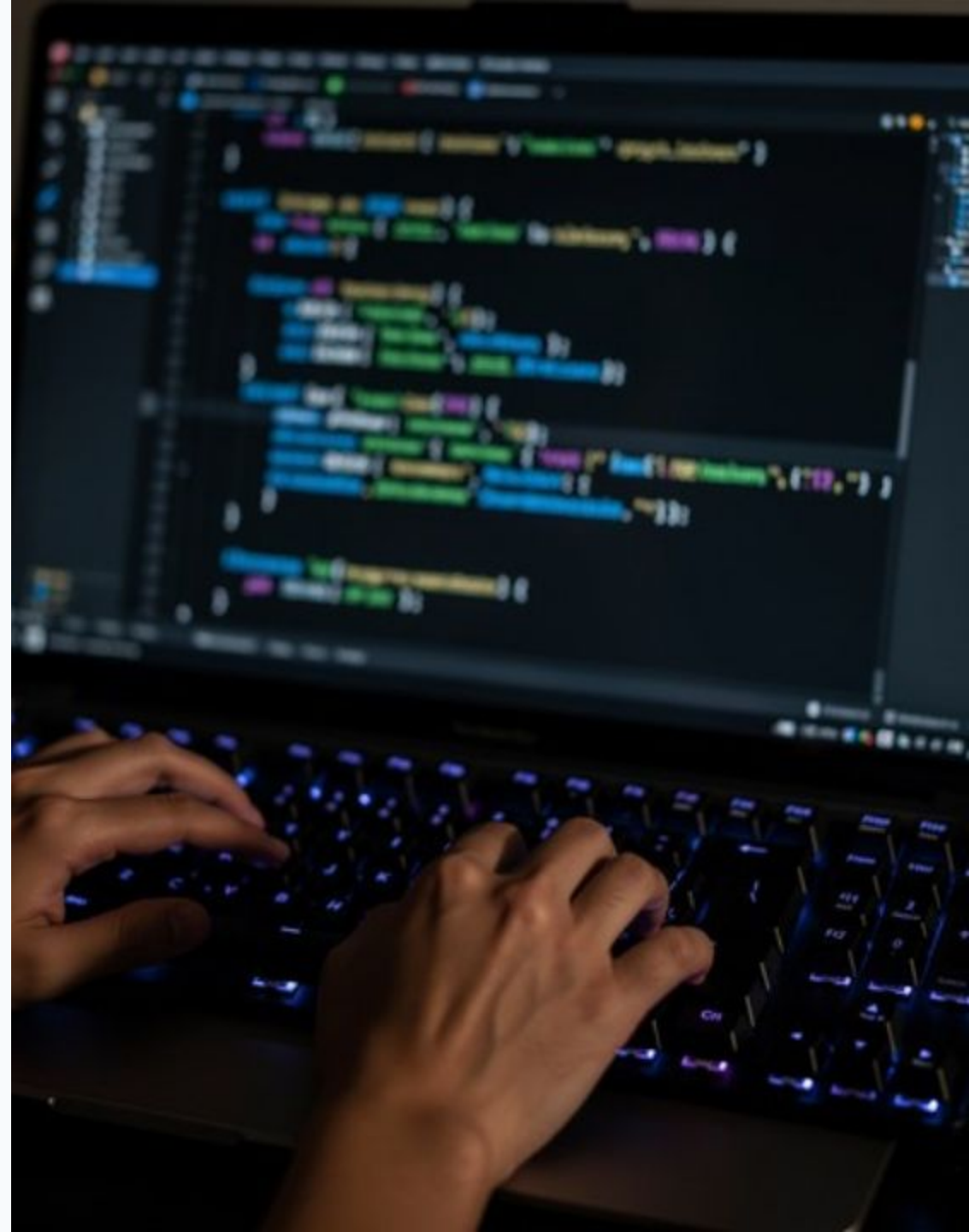
routes.py

application/

loan_service.py

infrastructure/

db_repository.py



Driver Arquitectónico

Mantenibilidad Extrema

"Cambiar el menú sin tener que demoler y reconstruir la cocina entera."



El Costo (Trade-Off)

Mayor complejidad inicial y ligera penalización de rendimiento por el paso estricto de mensajes.



Regla Estricta

"El mesero no puede entrar a la despensa a sacar ingredientes, incluso si es más rápido."

Evidencia en Préstamos Python



Rutas Aisladas

API independiente de la lógica de aprobación



Lógica Testeable

Pruebas al servicio sin levantar base de datos.



Datos Seguros

Consultas confinadas al repositorio.



Orden y Escalabilidad

Separar responsabilidades asegura que el sistema pueda crecer, probarse y modificarse sin miedo a colapsar.

¿Preguntas?

Image Sources



https://media.crayon.com/2026-06-12/rQG1CnzT_65nEwtyx_tOg.webp

Source: www.crayon.com



https://images.stockcake.com/public/0/3/5/0353a3e6-208b-49c4-9689-b6395c3d922a_large/fine-dining-elegance-stockcake.jpg

Source: stockcake.com



https://images.stockcake.com/public/5/7/b/57bfe6ac-e248-4b4d-b9ee-086d261513eb_large/night-programming-session-stockcake.jpg

Source: stockcake.com